Functional Programming in Java

Block 1: Functional Interfaces and Method References
(Student Exercise Sheet)

Course: Technology and Programming Paradigms
School of Engineering (EPI) — University of Oviedo

In the following eight exercises, you will work with some basic Java functional interfaces (`Predicate`, `Function`, `Consumer`, `Supplier`) and different forms of *method references*.

For each exercise, a specific functional interface is indicated, and several candidate method references are proposed (using the method reference notation, e.g., `String::isEmpty` or `Integer::parseInt`).

In each exercise you must:

- Choose, among the proposed methods, one that is compatible with the requested functional interface.
- Write the functional interface declaration using a method reference with the `::` notation.
- Write the equivalent declaration using a lambda expression (whenever possible).
- Add simple Java code showing how the abstract method works (for example, invoking `test`, `apply`, `accept` or `get`) and printing clear messages to the console.

Exercises

Exercise 1. Predicate<String> and Type Instance Method (String)

We work with the `Predicate<String>` functional interface and instance method references on type `String`.

The following list of possible method references is proposed:

- `String::isEmpty`
- `String::length`
- `String::toUpperCase`
- `String::trim`

1. Indicate which of the previous references is compatible with `Predicate<String>` (i.e., can be used to define a predicate on strings).
2. Declare a `Predicate<String>` using that method reference with the `::` notation.

3. Declare an equivalent `Predicate<String>` using a lambda expression.

4. Write test code creating at least two strings (e.g., an empty string and a non-empty one), applying both predicates using `test`, and printing messages to the console clearly indicating the result.

Exercise 2. Predicate<Character> and Static Method (Character)

We work with the `Predicate<Character>` functional interface and static methods of the `Character` class.

The following list of possible method references is proposed:

- `Character::isUpperCase`
- `Character::toUpperCase`
- `Character::charCount`

1. Indicate which of the previous references is compatible with `Predicate<Character>`.

2. Declare a `Predicate<Character>` using that method reference.

3. Declare an equivalent `Predicate<Character>` using a lambda expression.

4. Write test code calling the predicate with several different characters (e.g., uppercase and lowercase letters) and printing the results to the screen.

Exercise 3. Function<String,Integer> and Type Instance Method (String)

We work with the `Function<String,Integer>` functional interface and instance method references on type `String`.

The following list of possible method references is proposed:

- `String::length`
- `String::isEmpty`
- `String::toUpperCase`

1. Indicate which of the previous references can be used to define a `Function<String,Integer>`.

2. Declare a `Function<String,Integer>` using that method reference.

3. Declare an equivalent `Function<String,Integer>` using a lambda expression.

4. Write test code calling the function with several strings (e.g., `"java"` and `"functional"`) and printing the results returned by `apply` to the console.

Exercise 4. Function<String,Integer> and Static Method (Integer)

We work with the `Function<String,Integer>` functional interface and static methods of the `Integer` class.

The following list of possible method references is proposed:

- `Integer::parseInt`
- `Integer::compare`
- `Integer::max`

1. Indicate which of the previous references is compatible with `Function<String,Integer>`.
2. Declare a `Function<String,Integer>` using that method reference.
3. Declare an equivalent `Function<String,Integer>` using a lambda expression.
4. Write test code converting several numeric strings (e.g., `"10"` and `"123"`) to integers using `apply` and showing the results on the screen.

Exercise 5. Consumer<String> and Object Instance Method (System.out)

We work with the `Consumer<String>` functional interface and instance methods of a `PrintStream` object (in this case, `System.out`).

The following list of possible method references is proposed:

- `System.out::println`
- `System.out::printf`
- `System.out::close`

1. Indicate which of the previous references is compatible with `Consumer<String>`.
2. Declare a `Consumer<String>` using that method reference.
3. Declare an equivalent `Consumer<String>` using a lambda expression.
4. Write test code invoking the `Consumer` several times with different messages, so the console output clearly shows when the method reference version is being used and when the lambda version is being used.

Exercise 6. Consumer<String> and Object Instance Method (List)

We work with the `Consumer<String>` functional interface and instance methods of a `List<String>` object.

Suppose you have a list:

```
List<String> words = new ArrayList<>();
```

The following list of possible method references is proposed:

- `words::add`
- `words::isEmpty`
- `words::size`

1. Indicate which of the previous references can be used to define a `Consumer<String>`.
2. Declare a `Consumer<String>` using that method reference.
3. Declare an equivalent `Consumer<String>` using a lambda expression.
4. Write test code using both `Consumer`s to add several elements to the list and, finally, print the list content to the screen.

Exercise 7. Supplier<Double> and Static Method (Math)

We work with the `Supplier<Double>` functional interface and the static method `random` of the `Math` class.

The following list of possible method references is proposed:

- `Math::random`
- `Math::abs`
- `Math::max`

1. Indicate which of the previous references is compatible with `Supplier<Double>`.
2. Declare a `Supplier<Double>` using that method reference.
3. Declare an equivalent `Supplier<Double>` using a lambda expression.
4. Write test code invoking the `Supplier` several times using `get` and showing the generated values on standard output.

Exercise 8. Supplier<StringBuilder> and Constructor (StringBuilder::new)

We work with the `Supplier<StringBuilder>` functional interface and constructor references.

The following list of possible method references is proposed:

- `StringBuilder::new`
- `String::new`

- `StringBuilder::append`

1. Indicate which of the previous references can be used to define a `Supplier<StringBuilder>`.
2. Declare a `Supplier<StringBuilder>` using that method reference.
3. Declare an equivalent `Supplier<StringBuilder>` using a lambda expression.
4. Write test code obtaining two instances of `StringBuilder` from the `Supplier`, building two different text messages (using `append`), and printing them to the console.

Block 2. Basic Streams

In this sheet, you work with the Java `Streams` API at a basic level. The goal is for you to practice:

- How to create streams from simple data structures (lists and arrays).
- How to use terminal operations such as `forEach`, `count`, `anyMatch`, `allMatch` or `noneMatch`.
- How to chain simple intermediate operations such as `filter`, `sorted`, `distinct` or `limit`, followed by a terminal operation.

In all exercises, it is recommended to:

- Write console messages using `System.out.println` explaining what is being done.
- Test the code with various values to observe the behavior of each `stream` operation.

Exercise 1. Creating Streams and using `forEach`

Create a program where you:

1. Define an integer array with some values (e.g., `{1, 2, 3, 4, 5}`).
2. Define a list of strings with some names (e.g., `"Ana"`, `"Luis"`, `"Bea"`, `"Carlos"`).
3. From the array, create an `IntStream` using `Arrays.stream(...)`.
4. From the list, create a `Stream<String>` using the list's `stream()` method.
5. Traverse the `IntStream` using `forEach` and print each number to the screen, clearly indicating that this is the stream created from the array.
6. Traverse the `Stream<String>` using `forEach` and print each name to the screen, indicating that this is the stream created from the list.

Exercise 2. Terminal Operations: `count`, `anyMatch`, `allMatch`

Now work with a list of integers, for example:

```
List<Integer> nums = Arrays.asList(3, 5, 8, 10, 11);
```

From this list:

1. Create a `stream` and use `count()` to get the total number of elements. Show the result on the screen.
2. Create the `stream` again on the same list and use `anyMatch` with a lambda expression to check if any number is even. Show the result.
3. Create the `stream` again and use `allMatch` with a lambda expression to check if all numbers are positive. Show the result.
4. Add explanatory messages to the output to distinguish the result of each operation (`count`, `anyMatch`, `allMatch`).

Exercise 3. Terminal Operation `noneMatch`

Define a list of integers consisting only of odd numbers, for example:

```
List<Integer> nums2 = Arrays.asList(1, 3, 5, 7);
```

From this list:

1. Create a `stream` and use `noneMatch` with a lambda expression checking if a number is even.
2. Store the boolean result in a variable (e.g., `noneEven`).
3. Show the original list and the `noneMatch` result on the screen, clearly indicating what the obtained value means (e.g., "no element is even").

Exercise 4. Chain `filter` + `forEach`

Now work with a list of strings representing programming language names, for example:

```
List<String> words =
    Arrays.asList("java", "python", "c", "go", "haskell");
```

From this list:

1. Create a `stream` on the list.
2. Use `filter` with a lambda expression to keep only words whose length is greater than 3.
3. Then, use `forEach` to print the filtered words to the screen.
4. Add a message before the output indicating that "words with length greater than 3" are going to be shown.

Exercise 5. Chain `filter` + `sorted` + `forEach`

Define a list of strings with city names, for example:

```
List<String> cities =
    Arrays.asList("Oviedo", "Gijon", "Aviles", "Mieres");
```

From this list:

1. Create a `stream` on the list.
2. Apply `filter` to keep only cities whose names start with `'A'` or `'G'`.
3. Then apply `sorted()` to order the resulting cities alphabetically (natural `String` order).
4. Use `forEach` to print the cities that passed the filter, in the sorted order.
5. Write a message before the output indicating what will be shown (e.g., "Cities starting with A or G, sorted").

Exercise 6. Chain `distinct` + `filter` + `count`

Consider a list of integers containing repeated values, for example:

```
List<Integer> values =
    Arrays.asList(1, 2, 2, 3, 4, 4, 5);
```

From this list:

1. Create a `stream` on `values`.
2. Use `distinct()` to remove repeated values.
3. Apply `filter` to keep only even numbers.
4. Use `count()` as a terminal operation to count how many distinct even numbers there are.
5. Display the original list and the result of `count()` on the screen, with an explanatory message (e.g., "Count of distinct even numbers: ...").

Exercise 7. Chain `filter` + `limit` + `forEach`

Define a list of integers containing several values greater than 10, for example:

```
List<Integer> bigNums =
    Arrays.asList(10, 3, 7, 20, 25, 2, 30);
```

From this list:

1. Create a `stream` on `bigNums`.
2. Use `filter` to select only numbers greater than 10.

3. Apply `limit(2)` to keep only the **first two** numbers meeting the condition.
4. Use `forEach` to print those two numbers to the screen, with a message clearly indicating that these are "the first two numbers greater than 10 found in the list".

Block 3. Mini-Application: Mini Text Stream Lab

In this block, you will work with a console mini-application that uses `streams` to analyze a text. The Java skeleton provided to you already includes:

- A constant `TEXT_LINES` of type `List<String>` containing the base text lines.
- A helper method `getAllWordsFromText()` that returns a `List<String>` with all the words in the text.
- A helper method `containsDigit(String s)` that returns `true` if the string `s` contains any digit and `false` otherwise.
- A method `runMiniTextStreamLab()` that displays an interactive menu and calls the methods: `runOption1ShowOriginalText()`, `runOption2BasicStatistics(List<String>)`, `runOption3Word` `runOption4DistinctSortedWords(List<String>)`, `runOption5CheckWordConditions(List<String>)` and `runOption6FirstWordsWithMinLength(Scanner,List<String>)`.

Your task is to complete these methods so that, using the list of lines (`TEXT_LINES`) and the list of words returned by `getAllWordsFromText()`, you create the necessary `streams` and display the results indicated in each exercise.

In all exercises you must:

- Create the corresponding `stream` (e.g., using `words.stream()`).
- Chain the indicated intermediate operations (`filter`, `sorted`, `distinct`, `limit`).
- Always finish with a suitable terminal operation (`forEach`, `count`, `anyMatch`, `allMatch` or `noneMatch`).
- Write clear console messages using `System.out.println`.

Exercise 1. Show Original Text

Complete the method `runOption1ShowOriginalText()` so that you:

1. Create a `stream` from `TEXT_LINES`.
2. Traverse the `stream` with `forEach` and print each line of the text.

The output should include a header like `Original text (lines):` followed by each line of the original text.

Exercise 2. Basic Statistics on Words

Complete the method `runOption2BasicStatistics(List<String>)` so that you:

1. Calculate the total number of words with `count()`.
2. Calculate the number of distinct words with `distinct()` and `count()`.
3. Calculate how many words have length greater than 5 using `filter`.
4. Show the result of each calculation with messages like:
   - `Total words: ...`
   - `Distinct words: ...`
   - `Words with length >5: ...`

Exercise 3. Words Starting with a Given Letter

Complete the method `runOption3WordsStartingWith(Scanner,List<String>)` so that you:

1. Ask the user for a letter.
2. Create a `stream` from the list of words.
3. Use `filter` to select words starting with that letter (ignoring case).
4. Print the selected words with `forEach`.

If you enter an empty string, you must show a warning message and not execute the filter.

Exercise 4. Distinct Words Sorted Alphabetically

Complete the method `runOption4DistinctSortedWords(List<String>)` so that you:

1. Create a `stream` from the list of words.
2. Apply `distinct()` to remove duplicates.
3. Apply `sorted()` to sort alphabetically.
4. Print the words using `forEach`.

Exercise 5. Checking Conditions on Words

Complete the method `runOption5CheckWordConditions(List<String>)` so that you:

1. Use `anyMatch` to check if there is any word with length greater than 10.
2. Use `allMatch` to check if all words have length greater than 1.
3. Use `noneMatch` together with `containsDigit(String)` to verify that no word contains digits.
4. Show the results of these three checks.

Exercise 6. First Words with Minimal Length

Complete the method `runOption6FirstWordsWithMinLength(Scanner,List<String>)` so that you:

1. Ask the user for an integer representing the minimum length.
2. Create a `stream` from `words`.
3. Use `filter` to select words with length greater than or equal to the indicated one.
4. Use `limit(5)` to show only the first five words meeting the condition.
5. Print each resulting word with `forEach`.

This completes the *Mini Text Stream Lab* mini-application. When running the program and choosing the different menu options, the corresponding outputs for each exercise should be generated in the console.